Java Collections & HashMap Cheat Sheet

Java Collections Framework (JCF)

Why Collections?

Arrays have limitations:

- Fixed size
- No built-in insertion/deletion utilities
- Limited searching and sorting support

Collections provide:

- Dynamic size
- Built-in algorithms
- Standard interfaces
- Better performance for different use cases

Collection Hierarchy

```
Iterable
   |
Collection
   |
   |-- List
   |     |-- ArrayList
   |     |-- LinkedList
   |     |-- Vector
   |
   |-- Set
   |     |-- HashSet
   |     |-- LinkedHashSet
   |     |-- TreeSet
   |
   |-- Queue
         |-- PriorityQueue
         |-- ArrayDeque

Map (Separate Hierarchy)
   |
   |-- HashMap
   |-- LinkedHashMap
```

```
    |-- TreeMap
    |-- Hashtable
```

Why doesn't Map extend Collection?

- Collection stores individual elements.
- Map stores key-value pairs.

List vs Set vs Queue vs Map

Interface	Order	Duplicates	Typical Use
List	Yes	Yes	Playlist, API response, shopping cart
Set	Not guaranteed (HashSet)	No	Unique usernames, permissions, tags
Queue	FIFO	Yes	Task scheduling, message queues
Map	Key-Value	Keys: No, Values: Yes	Caching, lookup by ID

ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Internal Structure	Dynamic Array	Doubly Linked List
Random Access	O(1)	O(n)
Insert/Delete	O(n)	O(n) to locate + O(1) to link/unlink
Memory	Lower	Higher

Interview Point

LinkedList is **not faster for searching**.

Finding the node is still **O(n)**.

Only insertion/deletion **after reaching the node** is O(1).

Java LinkedList Optimization

LinkedList is doubly linked.

Java starts traversal from:

- Head if index < size/2
- Tail if index >= size/2

Still worst-case complexity remains **O(n)**.

HashMap

Why HashMap?

Fast lookup by key.

Instead of:

```
O(n)
```

HashMap provides average:

```
O(1)
```

HashMap Stores

```
Key -> Value
```

Example

```
101 -> Mohit
102 -> Rahul
```

Properties:

- Keys must be unique.
- Values can be duplicated.
- One null key allowed.
- Multiple null values allowed.

Internal Node Structure

```
Node {
    int hash;
    K key;
    V value;
    Node next;
}
```

HashMap put()

Steps:

1. Compute hashCode().
2. Calculate bucket index.
3. Bucket empty?
4. Yes → Insert.
5. No → Traverse bucket.
6. Compare hash.
7. Compare equals().
8. Same key?
9. Yes → Update value.
10. No → Add new node.
11. Bucket size > 8 (and capacity >= 64)?
12. Convert Linked List to Red-Black Tree.

HashMap get()

Steps:

1. Compute hashCode().
2. Find bucket.
3. Traverse nodes.
4. Compare hash.
5. Compare equals().
6. Return matching value.

---

hashCode() vs equals()

hashCode()

Used to determine the bucket.

equals()

Used to compare keys inside the bucket.

HashMap requires both.

HashMap Contract

If:

```
a.equals(b) == true
```

Then:

```
a.hashCode() == b.hashCode()
```

Must also be true.

Reverse is NOT mandatory.

Two different objects may have the same hashCode (collision).

Why Override Both?

Wrong

```
equals() ✔
hashCode() ✘
```

Result

Different buckets.

equals() may never be called.

HashMap lookup fails.

Correct

```
equals() ✔
hashCode() ✔
```

Both logically equal objects go to the same bucket and can be matched.

Hash Collision

Collision = Different keys produce the same bucket.

Before Java 8

```
Bucket
  |
Node
  |
Node
  |
Node
```

After Java 8

If:

- Bucket size > 8
- Table capacity >= 64

Linked List

↓

Red-Black Tree

Lookup improves from:

```
O(n)

↓

O(log n)
```

Initial Capacity

Default

```
16
```

Why?

HashMap always keeps capacity as a power of 2.

```
2
4
8
16
32
64
128
```

Reason:

Fast bucket calculation using:

```
(capacity - 1) & hash
```

instead of

```
hash % capacity
```

Bitwise AND is faster than modulo.

Load Factor

Default

```
0.75
```

Threshold

```
Capacity × Load Factor
```

Example

```
16 × 0.75 = 12
```

After roughly 12 entries, resize is triggered.

Why 0.75?

- Good balance between memory usage and collision rate.
- Avoids resizing too frequently while maintaining fast lookups.

Rehashing

When capacity increases:

```
16 → 32
```

Bucket positions change.

All existing entries are recalculated and redistributed into the new bucket array.

This process is called **Rehashing**.

HashSet Internals

HashSet is backed by a HashMap.

Internally:

```
HashMap<E, Object>
```

Example

```
Apple -> PRESENT
Banana -> PRESENT
Orange -> PRESENT
```

Only the keys matter.

This is why duplicate elements are not allowed.

Important Time Complexities

Operation	Complexity
ArrayList get()	O(1)
LinkedList get()	O(n)
HashMap get()	O(1) Average
HashMap put()	O(1) Average
HashMap Collision	O(n) before Java 8
Red-Black Tree Search	O(log n)

Most Important Interview Questions

1. Why doesn't Map extend Collection?
2. ArrayList vs LinkedList?
3. Why is LinkedList slower for random access?
4. Why is HashMap O(1)?
5. What is a hash collision?
6. Why override equals() and hashCode() together?
7. Can two objects have the same hashCode?
8. Yes.
9. Can two equal objects have different hashCodes?
10. No.
11. Why is initial capacity 16?
12. Why is load factor 0.75?
13. What is rehashing?
14. Why was Red-Black Tree introduced in Java 8?
15. How does HashSet work internally?

Interview Tips

- Don't say **"LinkedList is faster."**
- Say: **"It depends on the access pattern."**
- Don't say "`equals()` **compares objects."**

- Say: "`hashCode()` **determines the bucket, and** `equals()` **identifies the correct key within that bucket."**

- Remember:

- HashMap first uses **hashCode()**.
- Then it uses **equals()**.

Understanding this flow is the key to mastering HashMap.

Revision Summary

✅ Collections Framework

✅ Collection Hierarchy

✅ List vs Set vs Queue vs Map

✅ ArrayList vs LinkedList

✅ HashMap Internals

✅ Buckets

✅ hashCode()

✅ equals()

✅ Hash Collisions

✅ Linked List → Red-Black Tree

✅ Initial Capacity

✅ Load Factor

✅ Rehashing

✅ HashSet Internals